KlinikLab - Durable Next-Chat Handoff: Local Unreal Clinical Render First

Snapshot: 31 October 2026, Europe/Berlin. Evidence-first; no background task or future delivery is implied.
Repository: https://github.com/ajeshkg17/KlinikLab
Integration branch: integration/single-chat-validation-2026-10-30
Exact functional source SHA for the NEXT owner-PC attempt: 0f27a7e0e7179ef30a507e8bc0251749d6bbbad1
Source CI for that SHA: https://github.com/ajeshkg17/KlinikLab/actions/runs/41226730861
This Markdown/PDF publication adds a documentation commit after 0f27a7e... Do not confuse that documentation commit with the tested native renderer code. For the next owner-PC test, use the complete exact 40-character source SHA above, not the moving branch tip.

1. Binding order: LOCAL KLINIKLAB UNREAL, THEN QUEST

The user explicitly corrected the development order. FIRST produce an identifiable source-derived KlinikLab clinical education scene in the native Unreal Engine 5.8.4 editor on the owner Windows NVIDIA RTX 5060 workstation. Obtain a fresh genuine Unreal HighResShot, personally inspect its original image pixels, and corroborate the specific UnrealEditor process's D3D12 NVIDIA adapter selection and GPU 3D-engine work. ONLY AFTER the local native clinical content and process-specific GPU gate is independently accepted may development begin on Pixel Streaming 2, WebRTC/LAN, security/pairing, Quest 2 receiver, headset stereo, or controller return.

No false acceptance from GitHub CI, source vertex counts, unloaded scene, self-reported metadata, blank-project evidence, FFmpeg NVENC alone, Chrome WebGL/IWER, a diagnostic viewport, or a mock Unreal Python test. Do not claim clinically calibrated anatomy; the FDI16 tooth is an original, *schematic educational* implicit-solid model, not a patient scan.

2. Current status: SR1B NOT ACCEPTED

Implemented in repository: a source-pinned OBJ/MTL export from the real production KlinikLab volumetric-lab.js ToothSolid('molar','16') and VoxelMesher; an isolated UE5.8 D3D12 project; Unreal Python native importer/level script; a new direct-source ProceduralMeshComponent path; Windows Build/Open/Collect runner, hashes and native log/PNG/PID GPU counter collector; tests and roadmap/runbook.

Not yet observed: a successful owner-PC Stage 1 run on commit 0f27a7e..., any saved and independently reviewed native clinical Unreal image, persisted procedural triangle-index byte readback, process-specific NVIDIA GPU 3D activity during rendering, any native KlinikLab frame encoded or streamed, physical Quest 2 operation, or dental professional signoff.

New untested runtime risk: Unreal Python editor support for dynamically creating persistent ProceduralMeshComponent subobjects through SubobjectDataSubsystem, the precise UE5.8 Python signature of create_mesh_section_linear_color, material binding, serialization, saved-level reload, and pixel appearance. The new source/mocked regression cannot establish these on the owner GPU. If Stage 1 fails, preserve the original error and fix only the actual failed contract without relaxing input topology.

3. Precise failure chronology (all original evidence stays unchanged)

- Blank UE5.8 SR1 evidence, NOT KlinikLab: a read-only original native log SHA-256 3923f2e115a651597893e3d95c9d5c9a0755cf8c4a0a00ac322b2a75c9596e67 was reanalysed at adapter parser cd75663d7f32c678104ce508594552c083b3218e. Owner PC observed classification D3D12_INDEX_MAPPED_TO_MATCHING_NVIDIA_GPU, chosen D3D12 adapter 0, RTX 5060, no adapter blockers. Supplemental derived JSON SHA-256 a1ca8d73ba0b2d57a9c6669252578770595 37dc89a41d7aaf2ed6d55c43cd65c. Original collector report was not overwritten and its other evidence limitations persist. No actual clinical GPU raster or stream proven.
- First clinical source export and OBJ grouping failure, c23c85a7956a284546509a1b3a5fc8ede6c94594: C:\klab-sr1b-clinical-c23c85a7-20261031-1650. Export succeeded: 14,856 before, 14,936 after; imported Unreal OBJ split into five StaticMeshes/materials, contrary to the original one-StaticMesh assertion. No level or accepted clinical screenshot.
- Five-tissue assembly fix failure, 54e3e06bc253d75a1136fed81243050db4a8e495: C:\klab-sr1b-clinical-54e3e06b-20261031-1700. Native material interfaces were correctly identified, but

before enamel source 10,320 -> UE render LOD0 6,724; cementum source 4,105 -> UE LOD0 3,116. Strict face-parity gate blocked level creation; no clinical screenshot accepted.
- Conservative and direct StaticMesh reconstruction failure, exact source 57801f1c7f84cc8e561cac8fc99045cb14d8f6b5: detached worktree C:\klab-sr1b-worktree-57801f1c-20261031-174525; evidence folder C:\klab-sr1b-clinical-57801f1c-20261031-174525. Owner PC Unreal 5.8.4's imported *source MeshDescription* retained 10,320 enamel triangles, but native *render LOD0* remained 6,724 after disabling degenerate removal and relevant build options. Direct StaticMeshDescription reconstruction then produced just 6,721 rendered triangles and failed with exact error: "Direct native StaticMesh build still removed source OBJ triangles: enamel, source=10320, native=6721". The original detailed traceback/report is C:\klab-sr1b-clinical-57801f1c-20261031-174525\unreal-clinical-level-build.json. No native clinical level/screenshot or GPU trace completed. Never edit/reuse/rename this evidence.
- Source geometry audit: original OBJ before: enamel 10,320, dentin 112, pulp 207, cementum 4,105, caries 112; after: enamel 10,308, dentin 205, pulp 207, cementum 4,105, caries 111. Totals 14,856 / 14,936. Exact source audit found zero zero-area faces and zero exact duplicate unordered faces, with only two triangles below 1e-6 cm^2 doubled area in each of enamel and cementum. Thus thousands of missing UE rendered faces are NOT explained by thousands of exactly degenerate source triangles. The precise internal Unreal optimization/welding/precision mechanism is not independently determined.
- Diagnostic screenshot LIMIT: any earlier diagnostic viewport/blank-project screenshot is not an accepted image of a completed, source-provenanced clinical FDI16 native Unreal scene. It does not establish five material groups, visible before/after cut, preservation of native procedural buffers, the correct Unreal process GPU activity, or clinical correctness. The source/pixel acceptance gate is still open; the user explicitly disallowed treating the diagnostic viewport as accepted.

4. Latest source correction at exact 0f27a7e0e7179ef30a507e8bc0251749d6bbbad1

Reason for new route: UE5.8.4 repeatedly removed valid faces when compiling a StaticMesh render LOD (10,320 source -> 6,724 / 6,721). The correction does NOT lower expected topology counts.

Selected native rendering method: the isolated visual-qa/ue5_8-kliniklab-local/KlinikLab_SR1B_Local.uproject explicitly enables the ProceduralMeshComponent plugin. visual-qa/sr1b-unreal-build-clinical-level.py imports the OBJ to source-identify the five KL_ materials and record *lossy imported StaticMesh LODs only as diagnostics*. It separately reads the SHA-verified original OBJ triangle stream via sr1b_topology_source.py, creates exact per-face position/normal/index buffers with sr1b_procedural_buffers.py, and attempts to attach one native material-specific procedural mesh section for each of enamel, dentin, pulp, cementum and caries to each of two labeled native tooth actors. It preserves original face winding and uses two-sided tissue materials because the marching-tetrahedra source has inconsistent winding.

Expected source submission contract:

Tissue	BEFORE	AFTER simulated cut
Enamel	10,320	10,308
Dentin	112	205
Pulp	207	207
Cementum	4,105	4,105
Caries	112	111
Total source triangles	14,856	14,936

Each source triangle contributes 3 indices. Report renderPath must be NATIVE_PROCEDURAL_MESH_EXACT_OBJ_FACE_INDEX_BUFFERS. nativeProceduralSourceTrianglesSubmitted must be 14,856 before / 14,936 after. nativeProceduralSections must include five correctly named tissue sections per tooth; the saved level must reload with both actors and all sections/materials present, recorded in nativeProceduralReloadedSections. The old importedTrianglesLOD0 counts remain diagnostic only and are not substituted into procedural face acceptance.

Unproven beyond source submission: even successful level save/reload with section count and visible flag does not prove exact byte-for-byte persistence of the raw procedural triangle indices. The script explicitly records savedProceduralTriangleIndexReadbackVerified=false. Actual native viewport pixels, geometry continuity, before/after cut differences and GPU engine attribution still need review.

Tests at exact SHA: GitHub Actions run 41226730861 completed SUCCESS with 240 Node regression tests passed, 0 failures/skips, plus 14 additional checks passed. Source/mock tests include the actual

10,320 -> 6,724/6,721 StaticMesh loss fixture, strict original material/face counts, source OBJ rejection of malformed/degenerate faces, procedural buffers with 3 indices per source face and mocked native components. CI does not run the owner's UE5.8.4 editor or dedicated RTX 5060.

5. EXACT next owner-PC Stage 1 - new, separate run; do not reuse 57801f1c

The next action is Build ONLY, not Quest, not Pixel Streaming and not a reused capture. Run from Windows PowerShell, with Node and UE5.8.4 installed. Adjust the $repo path ONLY if the actual local Git checkout differs. Preserve old failed evidence directories.

```powershell
$repo = 'C:\Users\ajesh\OneDrive\study and learn\coworker\KlinikLab'
$sha = '0f27a7e0e7179ef30a507e8bc0251749d6bbbad1'
git -C $repo fetch origin integration/single-chat-validation-2026-10-30
if ($LASTEXITCODE -ne 0) { throw 'Git fetch failed' }
$stamp = [DateTime]::UtcNow.ToString('yyyyMMdd-HHmmss')
$worktree = "C:\klab-sr1b-worktree-$($sha.Substring(0,8))-$stamp"
$run = "C:\klab-sr1b-clinical-$($sha.Substring(0,8))-$stamp"
git -C $repo worktree add --detach $worktree $sha
if ($LASTEXITCODE -ne 0) { throw 'Worktree creation failed' }
Push-Location $worktree
try {
  & .\visual-qa\sr1b-run-local-kliniklab.ps1 -Mode Build -ExpectedSha $sha -EvidenceDir $run
  if ($LASTEXITCODE -ne 0) { throw "Native clinical Build failed: $run" }
  $report = Get-Content (Join-Path $run 'unreal-clinical-level-build.json') -Raw | ConvertFrom-Json
  $report | Select-Object status, renderPath, nativeProceduralSourceTrianglesSubmitted,
    nativeProceduralSections, nativeProceduralReloadedSections,
savedProceduralTriangleIndexReadbackVerified, errors | Format-List
} finally { Pop-Location }
Write-Host "Immutable SR1B run: $run"
```

If Build fails: retain the entire unique evidence folder untouched. Read unreal-clinical-level-build.json -> errors and traceback, and collect the original Unreal Project\Saved\Logs log. Return the exact new run path and exception; never disable per-tissue strictness, replace clinical procedural buffers with the lossy StaticMeshes, or use diagnostic screenshot as accepted evidence. This next Stage 1 is expected to expose any UE5.8.4 Python ProceduralMeshComponent API/serialization constraints.

Only if Stage 1 succeeds with status=NATIVE_LEVEL_CREATED_PENDING_GPU_AND_PIXEL_EVIDENCE and correct source submission/reload: open the SAME run in the native Unreal editor with the SAME pinned worktree:

```powershell
& "$worktree\visual-qa\sr1b-run-local-kliniklab.ps1" -Mode Open -ExpectedSha $sha -EvidenceDir $run
```

Frame BOTH native FDI16 tooth actors in Lit mode; verify visibly distinct before/cut shape, labels and tissues; take genuine Unreal HighResShot 1920x1080. Then, only while the actual editor remains running/rendering, run:

```powershell
& "$worktree\visual-qa\sr1b-run-local-kliniklab.ps1" -Mode Collect -ExpectedSha $sha -EvidenceDir $run
```

Collector must retain original PNG and log SHA-256, exact project/PID, D3D12 selected RTX 5060, real Windows GPU Engine(*)\Utilization Percentage engtype_3d nonzero samples tied to that editor PID, and acknowledge that GPU engine LUID -> physical GPU mapping requires corroboration. The development assistant must personally inspect original PNG pixels, not infer success from PNG signature/hash, metadata, source counts or a watcher declaration. Without all applicable evidence, record SR1B NOT_ACCEPTED.

6. Other project tracks and guardrails

Current application server/server.mjs is a local Node API/file server, not a dedicated GPU simulation server. engine.js, xr-procedure.js, volumetric-lab.js are browser WebGL/WebXR and browser-side

procedural geometry. Legacy Meta IWER/Playwright XR screenshots reflect Chrome-rendered emulation, NOT native Unreal/remote Quest video; processed IDs must never be rerun. User wants direct original pixel review, immutable evidence, exact SHAs and unique run IDs. The earlier browser FDI16 job10 did produce distinct before/after eye images, but short WebM decoding/accepted real procedure video remained unproven. The Dundee left-maxillary-molar third-party candidate is not cleared for right FDI16 redistribution, so the native SR1B slice uses only original production schematic geometry.

Sources to read in a replacement chat:
- Roadmap: visual-qa/PHASE_ROADMAP.md - current priority override wins over older Quest-first prose.
- Current native runbook: visual-qa/SR1B_LOCAL_KLINIKLAB_RENDER.md.
- UE project: visual-qa/ue5_8-kliniklab-local/KlinikLab_SR1B_Local.uproject.
- Exporter: visual-qa/sr1-local-kliniklab-export.mjs; importer: visual-qa/sr1b-unreal-build-clinical-level.py.
- Exact source face validator and procedural buffers: visual-qa/sr1b_topology_source.py, visual-qa/sr1b_procedural_buffers.py; five-tissue contract: visual-qa/sr1b_section_contract.py.
- Windows orchestrator: visual-qa/sr1b-run-local-kliniklab.ps1.
- Focused tests: tests/sr1b_procedural_buffers_test.py, tests/sr1b_topology_source_test.py, tests/sr1b_section_contract_test.py, tests/sr1b_local_native_project.test.mjs, tests/sr1b_local_kliniklab_export.test.mjs.
- Architecture destination (future, not current): visual-qa/SERVER_RENDERING_ARCHITECTURE.md.
- Previous blank GPU logs and failed clinical runs are different evidence classes; retain all originals, not in Git.

7. Replacement-chat resume and handoff provenance

1. Fetch branch HEAD and read THIS paired handoff + roadmap + current SR1B runbook. The pinned functional test SHA remains 0f27a7e0e7179ef30a507e8bc0251749d6bbbad1 unless new functionality was explicitly committed and revalidated; publishing a handoff is NOT such a functional update.
2. The only immediate physical step is a fresh owner-PC SR1B Stage 1 build of the exact ProceduralMeshComponent path at that SHA. Inspect the actual report; if it fails, fix the observed UE5.8 Python/runtime incompatibility, rerun source tests and use yet another unique local run. Do not ask the user to repeat known paths or earlier observations.
3. On successful build, capture and personally review a genuine native clinical Unreal frame and actual editor-PID GPU engine data; mark the gate passed only after the appropriate independent evidence. Do not begin Pixel Streaming or Quest until then.
4. Never misidentify diagnostic viewport, static source MeshDescription, separate FFmpeg NVENC, browser/IWER images, documentation PDF, or green CI as completed local native clinical rendering.
5. This Markdown and the PDF at the same base path are documentation mirrors, not a signed experimental result. Future material handoff edits must update and verify BOTH. The actual GitHub publication commit SHA should be obtained from the repository and stated outside the self-referential handoff.

Artifact base path: visual-qa/handoffs/KLINIKLAB_DEVELOPMENT_SR1_HANDOFF_2026-10-31 (both .md and .pdf).